Sentiment Inversion: An Implementation of a Transform and Feedforward Neural Networks

Abraham Sharum
Computer and Information Sciences Department
University of Arkansas – Fort Smith

April 29, 2025

Abstract

The following document covers the tasks of sentiment analysis, language generation and sentiment inversion. The approach combines two neural networks models: a BowTie feedforward network for the task of sentiment classification. The BowTie is used to filter more important information with the goal of effective classification. The other model is a transformer, with scaled attention to maintain context of a given sentence when inverting the sentiment. It uses a modified loss function that notes the difference in probability between the original and inverted sentence. This is to penalize low differences in order to exaggerate sentiment change. The implementation of a feedback loop between the two models will allow for refinement of both models as they would work in an iron sharpens iron fashion. The datasets used within this implementation are the free IMDB and WikiText datasets from HuggingFace. The overarching goal is to build a model that inverts sentiment utilizing the Python programming language and it's libraries. The model will eventually be integrated into some application whether it be a web-interface, discord bot, or desktop application.

1 Introduction

When talking about language it is very easy to conceptualize as humans, our brains allow complex tasks to be seen as menial. For example, if I were to tell you "it's raining cats and dogs" it wouldn't take much to draw the conclusion that there is a heavy amount of rainfall outside. Additionally, the sentiment in that sentence can be seen as negative, due to the fact that most people don't enjoy rainy days. These conclusions were most likely drawn quickly and subconsciously, but what if we were to break down the components of said interaction. What mental processing goes into arriving at such conclusions, and if possible, how would one go about flipping the sentiment of our given phrase? This will be the focal point of the following document, how can we analyze sentiment, remember it's context, and invert it?

2 Background

To introduce the idea of inverting sentiment through text generation, we first must tackle the challenge of identifying sentiment. A requirement for analysis is *text encoding*, or in simpler terms representing words in numerical forms. Across the years of NLP research, methods for text encoding have included: word embedding, contextual embedding, sentence embedding, word frequency, word distribution and much more [**?**]. These numerical values are then passed through a neural network with many parameters used to calculate an output. The output of the network is used to calculate an error which is used to slightly tweak the parameters to get the desired output. This is a very simple

approach to classifying sentiment, as research built on this idea more effective ways for solving this problem were developed. One example of this is the *BowTie feedforward neural network* shown in [**?**], this network compresses the size of each layer and later expands it. This allows for the network to filter out less important words for sentiment while also reducing computation time. The presented network was able to classify at the same accuracy as previous more advanced networks at over 30 times faster speed. This is accomplished all while maintaining a lower overall *cross-entropy loss* (the difference between the predicted distribution vs the true distribution). When it comes to language generation different methods are required. One major problem with feedforward neural networks attempting language generation is the lack of ability to remember context across long documents [**?**, 1]. New models such as *recurrent neural networks(RNN's)* were more effective, however these models also struggle with contextual memory[**?**]. Along came the *transformer model*, these models were made specifically for NLP tasks and are considered the best for language generation [2]. Such models implement the skill of attention into an **Encode-Decoder Network**, the specific model in [2] uses *Scaled Dot-Product Attention* which builds on Dot-Product attention. The attention function, which maps a query and a set of key-value pairs to an output each of which is a vector, is scaled using a softmax function. This creates a probability distribution used as weights for each value in the key-value pairing. This allows for accurate contextual memory as parts of each sequence have a weighted level of "attention" building strong relations across other sequences. An example of this in action is in **Figure 3:[2]** where the model is able to piece together "making more difficult" from the verb "making". The model also implements self attention, this version of attention seeks to find the important parts of the current sequence in relation to itself rather than other sequences. The general attention is applied in the encoder-decoder model, where the decoder's previous state is the query and the encoder's outputs are the keys and values. The self-attention is applied to each layer of the encoder-decoder separately. The encoder learns the relationship between the input and it's tokens, the decoder learns the relationship between the output and it's tokens. On English-French and English-German translation task, this model had a *BiLingual Evaluation Understudy (BLEU)* of 28.4 and 41.8 respectively. These scores are greater than the previous state-of-the-art model which scores of 26.36 and 41.29 respectively. The transformer has the ability to be parallelized allowing it to be trained in $\frac{1}{4}$ of the time compared to the previous state-of-the-art model[2].

3 Specification

For my implementation I intend to use the BowTie neural network for classification and a modified transformer for language generation. The idea is to pass the classification of a given prompt generated by the BowTie and the prompt itself to the transformer. The transformer will then generate a sentence with the sentiment flipped and the context roughly the same. I will use **multi-head attention** from [2] to increase training speed and better language generation. For the BowTie plan to use the **Bernouli Distribution** and **binary cross-entropy** presented in [**?**] to predict and evaluate a dual sentiment model, said sentiments being positive and negative.

$$\textbf{Attention}(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

$$\textbf{Ber} = f(k; p) = \begin{cases} p & \text{if } k = 1, \\ 1 - p & \text{if } k = 0. \end{cases}$$

$$P(\widehat{y} \mid X, w) \prod_{i=1}^{n} \textbf{Ber}[\widehat{y_i} \mid \textbf{sigm}(x_i w)]$$

$\widehat{Y}$ is a vector of predictions such that $\forall \widehat{y}_i \in \widehat{Y}\ \exists[0:1]$
Q, K and V are matrices of queries, keys and values respectively.
X is a matrix of row vectors x_i named regressors and w are the regression weights, ϵ is an error[2, **?**].

I aim to fine tune the BowTie network for higher accuracy so that the transformer will have more reliable input for it's task, I also want to attempt to create a feedback loop between the networks. If implemented correctly this will allow for the BowTie to classify the sentiment of the generated sentence. The difference of probability between the original and new sentence can used as an evaluation metric to determine how exaggerated the sentiment flip was.

I plan to use the **huggingface**: **wikitext** and **IMDB** datasets for language generation and sentiment analysis respectively. The IMDB data set will require modification for the use of as each entry will need it's counterpart sentiment for accuracy, however the need for this paired data can be averted if the feedback loop is implemented. If it cannot be implemented then a pre-trained model will be used to flip the sentiments of each entry in the data creating a new entry, effectively doubling the IMDB dataset size. The inspiration for this idea is from [3]

The applications of interest are: web-interface, discord bot, desktop application with a GUI interface. The priority of which ones to be implemented reflects the manner in which they are listed.

4 Implementation

The algorithim planned is a training loop which uses the difference in the Bayesian probability between the original and generated sentence. I plan to implement this difference into the loss function to penalize low differences in hopes of generating a bigger difference in sentiment. However, the difference cannot be to high other wise the risk of a sentence with just positive or negative words. The programming language of choice is Python for the myriad of available libraries and cellular testing for incremental progress. I intend to use the PyTorch, NumPy, Pandas, and HuggingFace frameworks. For text encoding I plan to use tokenezation methods from the HuggingFace transformers library [1].

References

[1] Patrick S. H. Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *CoRR*, abs/2005.11401, 2020.

[2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017.

[3] Zaitian Wang, Pengfei Wang, Kunpeng Liu, Pengyang Wang, Yanjie Fu, Chang-Tien Lu, Charu C. Aggarwal, Jian Pei, and Yuanchun Zhou. A comprehensive survey on data augmentation, 2024.